Neuromorphic DHT — NH-1

A Learning-Adaptive Distributed Hash Table

with Axonal Publish-Subscribe

Research brief · 25 K-node benchmark · v0.3

David A Smith · YZ.social
davidasmith@gmail.com

Source, data, and simulator: `github.com/YZ-social/dht-sim`

What is a DHT? Why we need them

A **distributed hash table** is the foundation for finding *anything* in a decentralised network: any node, given a key, can locate the value in O(log N) hops with **no central authority, no privileged peer, no trusted coordinator**.

It is the substrate beneath:

- Decentralised pub/sub and messaging — every secure group chat, every federated feed
- Content-addressable storage (IPFS, Filecoin)
- Peer discovery in BitTorrent, Ethereum, Mastodon federation
- Public-key directories without certificate authorities
- Decentralised databases, namespaces, identity systems

Why we *desperately* need them now:

Centralised platforms have become single points of surveillance, censorship, and outage. Secure communication can no longer assume a trustworthy intermediate. Sovereignty over identity, location, and routing requires a substrate that **no one party can compromise**.

A DHT is the minimum viable substrate. Without it, every "decentralised" service is one trusted server away from being centralised again.

The Neuromorphic DHT — how it differs

Traditional DHTs (Kademlia, Pastry, Tapestry) treat the routing table as a **static data structure**: fixed buckets, one rule for replacement, no awareness of the traffic flowing through. They were designed in 2001-2002 for stable nodes — and they do not adapt to the network they live in.

The **Neuromorphic DHT** treats every peer as a *synapse*: a learnable edge with a weight that grows with successful traffic and decays without it. Routing consults those weights. Eviction picks the least-vital edge. The table changes as the network changes.

	Traditional DHT	Neuromorphic DHT
Routing table	Fixed K-bucket per stratum	Weighted synaptome
Edge state	Live / dead	Weight ∈ [0, 1], recency, locked-on-use
Routing decision	Greedy XOR	Action Potential (XOR × weight × latency)
Adapts to traffic?	No	Yes — LTP, triadic closure, hop caching
Adapts to churn?	Lazy bucket refresh	Active dead-peer eviction + temperature reheat
Pub/sub	Layered on top	Integrated as axonal delivery trees

The trade is "fixed and analytical" → "adaptive and empirical". The whitepaper-correctness of K-buckets gives way to *measured* behaviour — which is why the rest of this talk is about measurement.

Pub/sub belongs in the network

The end-to-end argument (Saltzer, Reed, Clark, 1984): functions that can be implemented at the endpoints should be implemented at the endpoints. Put nothing in the network that doesn't *have* to be there.

Pub/sub routing has to be there. A subscriber and a publisher cannot, end-to-end, discover the multicast paths that connect them — only the network can. Treating pub/sub as a separate "application overlay" forces it to reproduce the routing layer with no actual visibility into it. The end-to-end principle applies to *meaning*; **delivery is a network function**.

A decentralized DHT is the right place for it. Every node is *both* an endpoint and a relay. The mechanism is content-blind: it routes bytes toward a topic, with no inspection and no control over what travels.

Consequences:

- **Everyone becomes a broadcaster** — peer-symmetric, no privileged position, no central authority.
- The network is **not smart**. It has no control over the information it carries.
- It is a **protocol and a platform** for relaying that information.
- It is **hyper-connected**, within the physical limits of a phone or a browser.

This is the architectural commitment NH-1 inherits from the start. The whole DHT exists to make this delivery primitive viable.

Why a simulator

The environment our system runs on is **the Internet**.

We have attempted to build as fair and realistic environment simulation as we could. The interface and services it provides to system components will be identical to those of an actual deployed system.

This matters because after launch there is no easy way to monitor, repair or improve the network we deployed. By design we *can't*. The only honest path is a simulated Internet large enough for us to have a reasonable expectation of *verisimilitude* to study.

And everyone can see it. Source, data, every CSV in this deck — all open. This isn't for the faint of heart.

"As close to the actual system as possible" does not mean good. It means: this is what we have right now. We will continue improving it.

Why do we do this? Because we *can't* track how information flows through the real system. We can't understand it — much less figure out how to repair it or improve it — without the ability to simulate it.

The lab bench

Purpose-built simulator · ~25 K lines of JavaScript · open-source.

Modelled with fidelity

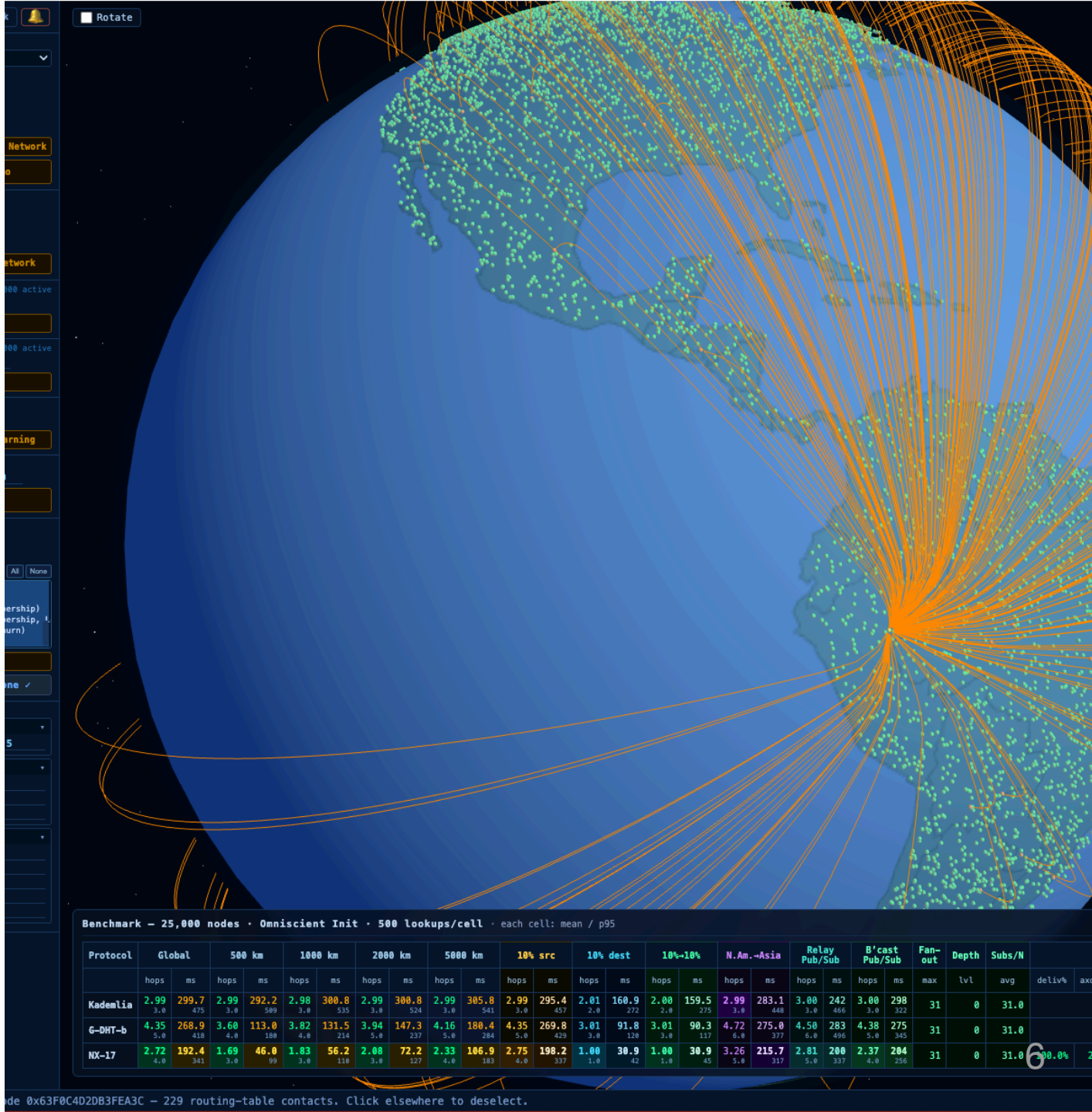
- GeoJSON land mask; haversine distances
- Up to 50 000 nodes on a navigable 3-D globe
- Per-hop simulated latency including 10 ms transit cost
- Bilateral connection caps mirroring WebRTC limits
- Every hop, ACK, reroute captured

Abstracted

- No wall-clock transport or encryption
- In-process node identity

Reproducibility

- All protocols build from the *same* seeded node set
- CSV export per run — every number in this deck is from `results/*.csv`



Naming as compact vocabulary

Four terms from neuroscience, each mapping to a specific data structure.

Term	Maps to
Synapse	One directed routing edge with a learned weight ∈ [0, 1]
Synaptome	The full edge set at a node — bounded at 50, a safe cross-browser WebRTC target
Neuron	A node: synaptome + temperature + message handlers
Axon	A directed delivery tree for one pub/sub topic, grown by routed subscribe

The vocabulary is descriptive, not metaphorical. Every term has one and only one corresponding artefact in the source code.

Protocols we will examine

We benchmark four DHTs at 25 K nodes under identical conditions. Each builds on its predecessor:

Protocol	What it adds	Inherits from
K-DHT (Kademlia, 2002)	XOR distance metric, K-buckets, α-parallel lookup	—
G-DHT (geographic)	S2 cell prefix in node IDs ⇒ regional locality	K-DHT
NX-17 (predecessor SOTA)	18 specialised rules, peak performance under tight cap	G-DHT *(via NX-1 … NX-15)*
NH-1 (this work, 2026)	Vitality-driven synaptome, unified admission gate	**NX-17** *(consolidation)*

NH-1 is *not* a fresh parallel design — it is the result of a careful analysis of NX-17 and every protocol before it. Each NX-17 rule was studied for what it does, why it was added, and whether its work could be folded into a smaller surface area.

- **NX-17** carries the lineage: 18 specialised rules, 44 parameters, ~2300 lines.
- **NH-1** consolidates that lineage: 12 rules, 12 parameters, ~270 lines — every admission decision through a single vitality score.

*We selected NH-1 as the deployment target for its **maintainability and understandability**. We continue to use NX-17 as the reference benchmark — the bar that NH-1 should approach, and that future work should match or surpass.*

K-DHT — Kademlia, the foundation

Distance metric: XOR — `d(a, b) = a ⊕ b`
Routing table: Every node maintains K peers per bucket. Lookup is a greedy walk toward the target in XOR distance, with α=3 parallel queries and K=20.
Properties: O(log N) hops in steady state. Static, predictable, analyzable.

Limit	Evidence at 25 K
No locality awareness	500 km lookup = **510 ms** — identical to its 499 ms global lookup
Fixed buckets	Same K peers regardless of usefulness; no response to traffic
Lazy churn repair	Broken edges persist until next bucket refresh
Broadcast cost O(audience)	Each pub/sub recipient reached by an independent lookup

K-buckets were a 2002 answer to "what's a stable routing table?" — static, predictable, analyzable. We will argue that adaptive weighting does better in practice.

The data structure is frozen. The network is not.

G-DHT — adding geographic locality

The change: `nodeId = S2 cell prefix (8 bits) ‖ H(publicKey)`.

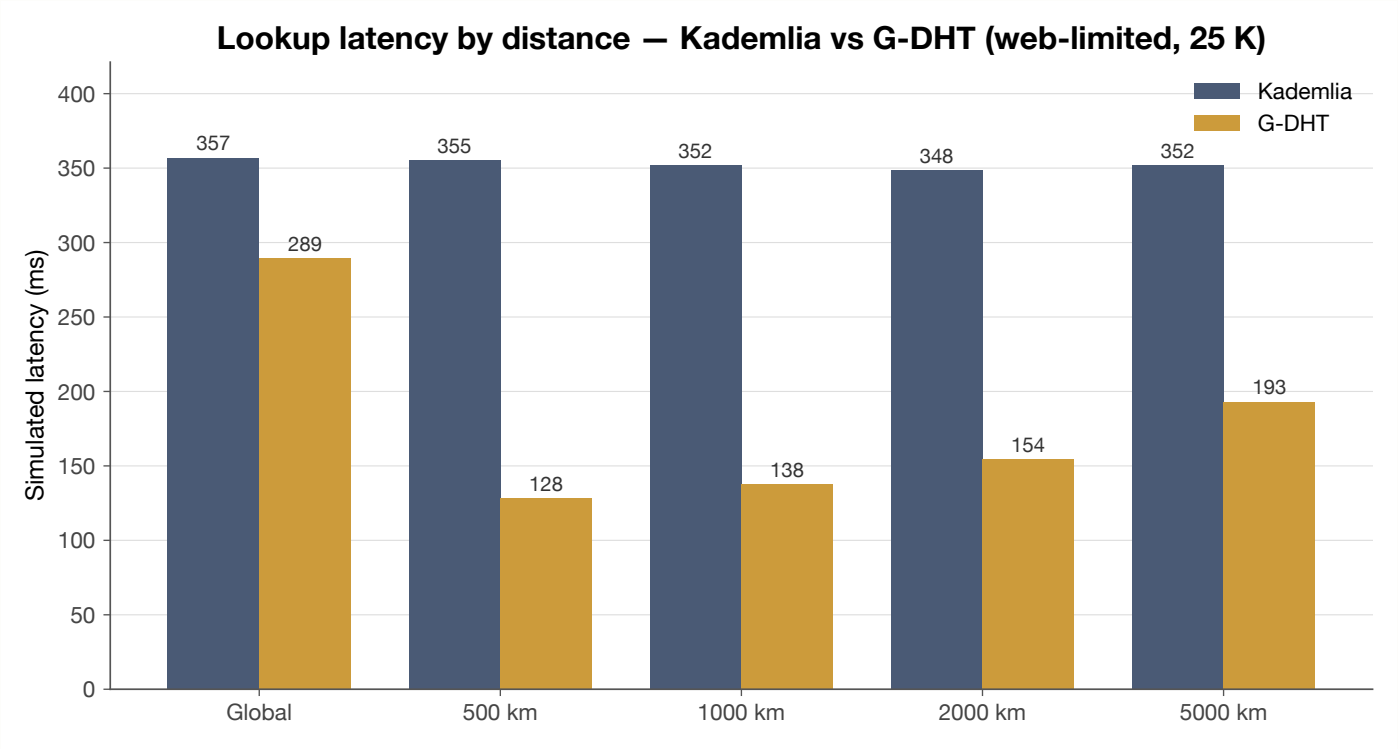
XOR in the ID space now approximates XOR in physical distance — the prefix dominates. Same K-bucket routing as Kademlia, no other changes.

Result at 25 K:

- 500 km regional latency: 510 ms → **150 ms** (3.4× faster)
- Global latency: 498 ms → **287 ms**

But still a *static* routing algorithm. No learning. No dynamics. The geographic prefix is a one-time topology decision, not an ongoing adaptation. Pub/sub is still bolted on top via K-closest replication, which drifts under churn.

G-DHT — locality at a glance



Lookup latency by distance, web-limited 25 K. G-DHT's geographic prefix gives regional traffic a 3-4× speedup over Kademlia. Global lookups still pay the full hop cost.

NH-1 — our chosen design

The change: every peer becomes a *synapse* with a learnable weight. The routing table is no longer fixed K-buckets — it is a **synaptome** that adapts to traffic, prunes by vitality, and grows via learning rules (LTP, triadic closure, hop caching).

The architectural arc:

- **Inherits** the entire NX lineage (NX-1 → NX-17): K-DHT's XOR metric, G-DHT's geographic prefix, NX-17's AP routing, LTP, triadic closure, hop caching, axonal pub/sub
- **Consolidates** five separate admission mechanisms (stratified eviction, two-tier highway, stratum floors, synaptome floors, adaptive decay) into a single vitality gate
- **Removes** rules whose contribution didn't measurably justify their parameter footprint
- **Keeps** every behavioural surface that NX-17 measurably needed

Design priority: simplicity.

	NX-17	NH-1
Rules	18	**12**
Parameters	44	**12**
Lines of code	~2300	**~270**
Admission gate	5 mechanisms (stratified eviction, two-tier, floors, adaptive decay…)	**1** (`_addByVitality`)

We accept a small performance gap against NX-17 in exchange for an architecture that one engineer can hold in their head — and that future engineers can extend without archeology.

NX-17 — the reference

Two research generations led here.

Launch pad: N-1 → N-15W. The original neuromorphic DHT family — fifteen-plus variants over years — explored the design space: synapses with weights, activation potentials, simulated annealing, hop caching, lateral spread, dendritic relay trees. Each pushed on one mechanism and exposed two new failure modes. By N-15W we had a working but tangled protocol with too many entangled rules to refactor in place.

Focused iteration: NX-1 → NX-17. The NX series re-grounded the architecture from a clean base. Each generation (NX-1, NX-2W, NX-3, … NX-17) addressed *one* concrete failure observed in its predecessor — one rule at a time, each measured against a benchmark. **NX-17 is the result.**

NX-17 is *better* than NH-1 under tight connection caps:

Test (25 K, cap=100)	NX-17	NH-1	Δ
Global lookup	**4.40 hops / 242 ms**	5.15 / 263 ms	+9 % ms
500 km lookup	**2.75 / 80 ms**	3.28 / 95 ms	+18 % ms

But it carries 18 specialised rules, 44 parameters, and ~2300 lines. Each rule earned its keep against a measured failure mode. Together they form a system hard to understand and hard to extend.

*We use NX-17 as the **reference benchmark**. The full rule list — and NH-1's disposition for each — appears later in this deck.*

NH-1 in one slide — five operations, one vitality model

NH-1 collapses the entire protocol into **five operations**, each scored by a unified vitality function.

Operation	What it does
NAVIGATE	AP routing + 2-hop lookahead + iterative fallback
LEARN	LTP, hop caching, triadic closure, incoming promotion
FORGET	Continuous decay + vitality-based eviction
EXPLORE	Temperature annealing + epsilon-greedy first hop
STRUCTURE	Stratified bootstrap + mixed-capacity (highway) deployment

```
vitality(syn) = weight × recency
```

A single `_addByVitality()` admission gate replaces stratified eviction, two-tier highway management, stratum floors, and synaptome floors. **~12 parameters** end-to-end.

What we call vitality

NH-1 admits and evicts every synapse via one scalar:

```
vitality(syn) = weight × recency(syn)
```

weight ∈ [0, 1] — trained by Long-Term Potentiation; reinforced on successful routing paths; decayed each tick.

recency = `exp(−Δepoch / RECENCY_HALF_LIFE)`; locked to 1.0 during inertia (LTP protection window).

The two factors are conventional individually. Hebbian potentiation (Hebb, 1949) gives the weight; exponential decay since last use (Ebbinghaus, 1885; LRU-K caching, O'Neil 1993) gives the recency. The closest biological analog is **synaptic tagging and capture** (Frey & Morris, *Nature* 1997) — synapses with both recent activity *and* sufficient potentiation are preferentially retained.

*The contribution of NH-1 is not the term or the formula. It is the use of this product as a **single admission gate** replacing five specialised mechanisms in NX-17: stratified eviction, two-tier highway management, stratum floors, synaptome floors, and adaptive decay.*

NH-1 rules — the full set, by operation

#	Rule	Operation	Why this rule
1	Stratified bootstrap	STRUCTURE	Cold-start coverage across all XOR distances
2	Mixed-capacity (highway%)	STRUCTURE	Realistic deployment — some peers run on real servers
3	AP routing	NAVIGATE	Learned-weight greedy walk dominates pure XOR
4	Two-hop lookahead	NAVIGATE	One probe of α second-hop options unblocks dead ends
5	Iterative fallback	NAVIGATE	If no progress, expand to k-closest from the synaptome
6	Long-Term Potentiation	LEARN	Reinforce edges on successful paths
7	Triadic closure	LEARN	If A→B→C succeeds twice, learn A→C
8	Hop caching	LEARN	Intermediate nodes cache the path for the destination
9	Incoming promotion	LEARN	Peers that contact me often become outbound synapses
10	Vitality-based eviction	FORGET	Drop the least-vital synapse when capacity is reached
11	Temperature annealing	EXPLORE	Cool exploration rate over time; reheat on dead-peer discovery
12	Epsilon-greedy first hop	EXPLORE	Small probability of random first hop avoids local minima

Every rule has a measured contribution at 25 K nodes. Ablations in Appendix C of the whitepaper.

STRUCTURE — bootstrap and deployment realism

Stratified bootstrap. Each new node is seeded with peers covering all XOR strata uniformly. Without this, the cold-start synaptome is dominated by lucky neighbours — local hops form, long hops don't.

Mixed-capacity deployment ("highway %"). Real P2P networks are heterogeneous: most peers are browsers (WebRTC ~100 connections), some are server-class (effectively unlimited).

A configurable `highwayPct` fraction of nodes is promoted to **server-class**: they accept unlimited inbound, hold a synaptome of up to 256, and act as transit hubs. The rest stay browser-class with the standard 50-synapse cap.

*15 % highway nodes captures **70 % of the available latency improvement** over the all-browser baseline (chart later in this deck).*

NAVIGATE — Action Potential routing

Each hop, score every candidate by a learned function:

```
AP(syn, target) = progress(syn, target) × syn.weight × ½^(latency_ms / 100)
```

- **progress** = XOR distance reduction toward target
- **weight** = LTP-reinforced [0, 1]
- **latency penalty** = exponential — fast peers preferred at all distances

Two-hop lookahead. When no first-hop is decisively best, probe the top α second-hop candidates and pick the path with the highest combined score.

Iterative fallback. If AP returns no candidate (every neighbour is "wrong direction"), fall back to k-closest-from-synaptome and retry. This rescues lookups that would otherwise stall in dead corridors.

LEARN — four reinforcement mechanisms

LTP (Long-Term Potentiation). When a lookup succeeds, every synapse on the successful path gets a weight bump and an inertia lock. Locked synapses cannot be evicted.

Triadic closure. If a node sees the same `(prev → next)` transit twice in a small window, it asks: "do I know prev directly?" If not, it adds a synapse — the network learns useful long edges from observed traffic.

Hop caching. Each intermediate node on a successful lookup adds the *destination* to its synaptome. The path becomes shorter on the next lookup to the same region.

Incoming promotion. When a peer reaches out to me repeatedly via incoming synapses, I promote it to a real outbound synapse — passive learning of who's interested in me.

FORGET — the unified admission gate

`_addByVitality(node, newSyn)` is called for *every* synapse addition: bootstrap, LTP, triadic, promotion, hop caching, annealing.

```
1. If synaptome has room → add.
2. Otherwise, find the lowest-vitality non-locked synapse.
3. If new synapse's vitality > victim's → swap.
4. Else → refuse silently.
```

This single function replaces:

- NX-17's stratified eviction
- NX-17's two-tier (synaptome + highway) management
- NX-17's stratum floors
- NX-17's adaptive decay parameters

Continuous decay (γ = 0.995/tick) erodes weight uniformly. Under-used synapses lose vitality and become eligible for replacement; well-used ones stay locked.

EXPLORE — temperature and epsilon

Temperature annealing. Each node carries a temperature `T ∈ [T_min, T_init]`. Cool by `T *= 0.9997` each lookup. Higher T → more probabilistic synapse selection (Boltzmann-style); lower T → greedy AP scoring.

Reheat on dead-peer discovery. When routing finds a dead peer, spike `T = max(T, 0.5)`. Accelerates exploration to repair damage.

Epsilon-greedy first hop. With probability ε = 0.05, replace the first AP-selected hop with a random synaptome member. Cheap insurance against early lock-in to a suboptimal corridor.

Both exploration mechanisms are biased toward learning rather than fully random — the floor is uniform sampling over the *current synaptome*, not over the network.

Pub/sub on top — axonal trees

NH-1 inherits NX-17's pub/sub stack verbatim. Three pieces:

Publisher-prefix topic IDs. `topicId = publisher.cellPrefix(8b) ‖ hash56(event_name)`. Both publisher and subscribers derive the same ID deterministically — no cross-party drift under churn.

Routed axonal tree. Subscribe is a routed message toward `topicId`. The first live axon role on the path intercepts. If none exists, the terminal node opens a new role and becomes root.

Self-healing via re-subscribe. Every role re-issues a subscribe on its 10-second refresh interval. The walk lands on a live axon, regardless of parent reorganisations.

*100 % delivery baseline; **100 % recovered delivery** under 5 % churn at 25 K nodes (this session's measurements).*

Temporal pub/sub — subscribe is a request for *history*

A subscribe is not just "send me future messages on this topic". It is **"send me every message I haven't already seen"**. Each subscribe carries a `lastSeenTs` — the highest publish timestamp this subscriber has observed.

Every relay node keeps a bounded ring buffer. When an axon role receives a publish, it records `{ json, publishId, publishTs }` in a local cache (capacity ≈ 100 messages — tunable per topic).

On subscribe arrival, the relay filters its cache to `publishTs > lastSeenTs` and replays the missed messages as a single batched message before forwarding the subscribe upstream.

Why this matters under churn:

The decentralised axon tree means every re-publish node — not just the publisher — holds a copy of recent history. If a parent dies and a subscriber's re-subscribe lands on a different live relay, that new relay can fill the gap from *its own* cache. **Healing and replay are the same mechanism.** No central log, no separate recovery RPC, no "catch-up" protocol.

A subscribe message in NH-1 is simultaneously a liveness probe, a tree-attach request, and a request for missed history. Three jobs, one envelope — that's the axonal healing model.

End-to-end tick

```
on lookup(target):
  hop = 0
  while hop < MAX_HOPS:
    candidates = synaptome ∪ incomingSynapses
    next       = AP_score(candidates, target)
    if next == null: next = iterative_fallback(target)
    if hop == 0 and rand() < ε: next = random(synaptome)

    record_transit(prev, next)         // LEARN: triadic
    promote_incoming_if_warranted()    // LEARN: incoming
    hop_cache_destination()            // LEARN: hop caching
    anneal_replace_lowest_vitality()   // EXPLORE
    hop++

on success: reinforce_path()           // LEARN: LTP
periodically: decay_all_weights()      // FORGET
```

Every operation cost is bounded: O(synaptome.size). At 50 synapses the per-hop compute is ~0.2 ms — small relative to the 10 ms transit cost.

Methodology

- **500 lookups per test cell.** Global, regional radii (500 / 1k / 2k / 5k km), pub/sub
- **Same node geometry** across all four protocols — direct comparison, not three independent builds
- **Bootstrap init** for production realism: sponsor-chain join + warmup. Omniscient init shown separately as a theoretical ceiling.
- **Churn** induced discretely (instantaneous kill) and continuously (1 % every 5 ticks)
- **Connection cap** 100 (browser-class, web-limited model)

Every number in the next section comes from a single 25 K-node CSV produced by the simulator at v0.67.02.

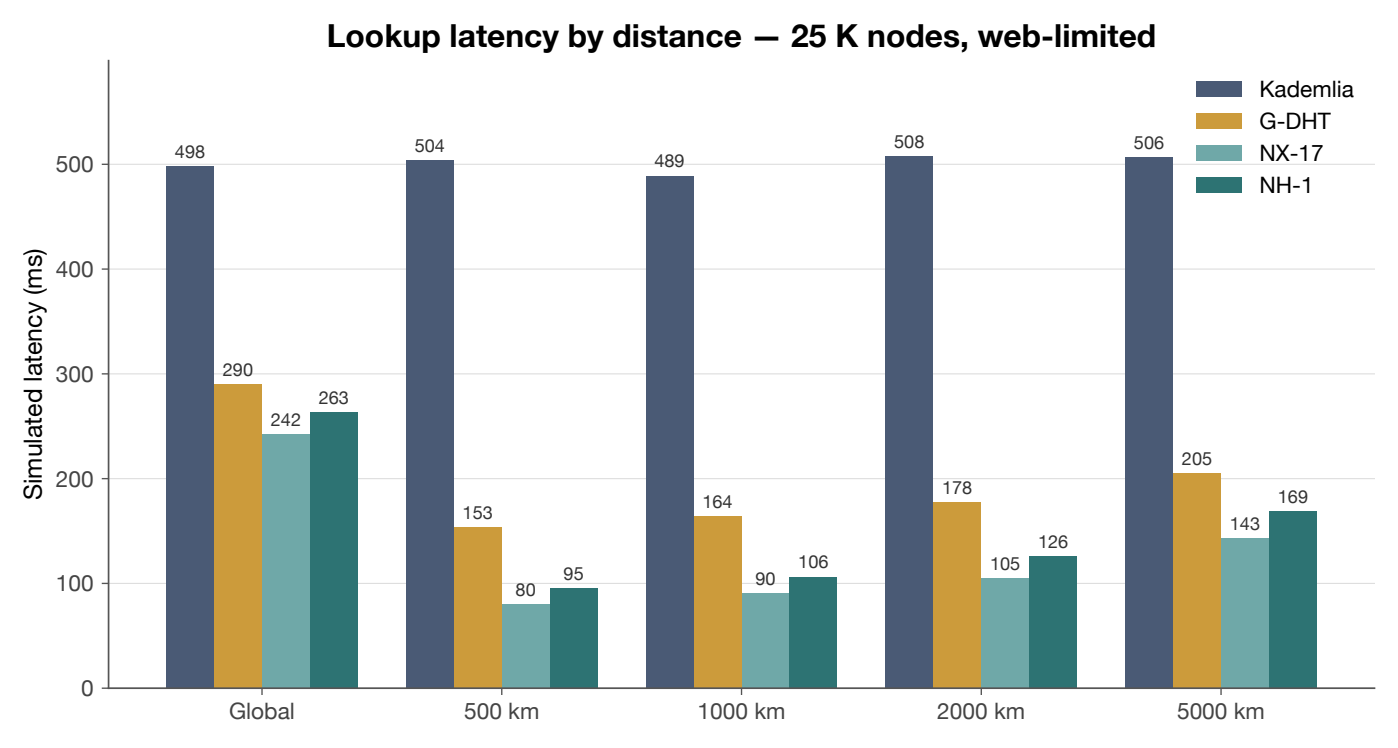
Four-way comparison at 25,000 nodes

Web-limited (cap=100), bootstrap init, geoBits=8, no highway promotion. Same node geometry across all four protocols.

Test	Kademlia	G-DHT	NX-17	NH-1
Global (hops / ms)	4.51 / **498**	5.60 / 290	4.40 / **242**	5.15 / 263
500 km (hops / ms)	4.51 / 504	4.91 / 153	2.75 / **80**	3.28 / 95
2000 km (hops / ms)	4.52 / 508	5.27 / 178	3.27 / **105**	4.01 / 126
5000 km (hops / ms)	4.47 / 506	5.46 / 205	3.74 / **143**	4.54 / 169
pubsubm delivered	n/a	n/a	**100 %**	**100 %**
pubsubm + 5 % churn (recovered)	n/a	n/a	**100 %**	**100 %**
dead-children / orphans	n/a	n/a	**0 / 0**	**0 / 0**

Both NX-17 and NH-1 dominate Kademlia and G-DHT on every distance band. NX-17 retains a small lead at the cap=100 ceiling — see "NH-1 vs NX-17" later in this deck.

Four-way comparison — at a glance



Lookup latency (ms) per distance band, 25 K nodes web-limited. NX-17 (light teal) and NH-1 (deep teal) cluster well below Kademlia (slate) and G-DHT (amber) at every distance.

Highway %: deployment-realistic knee

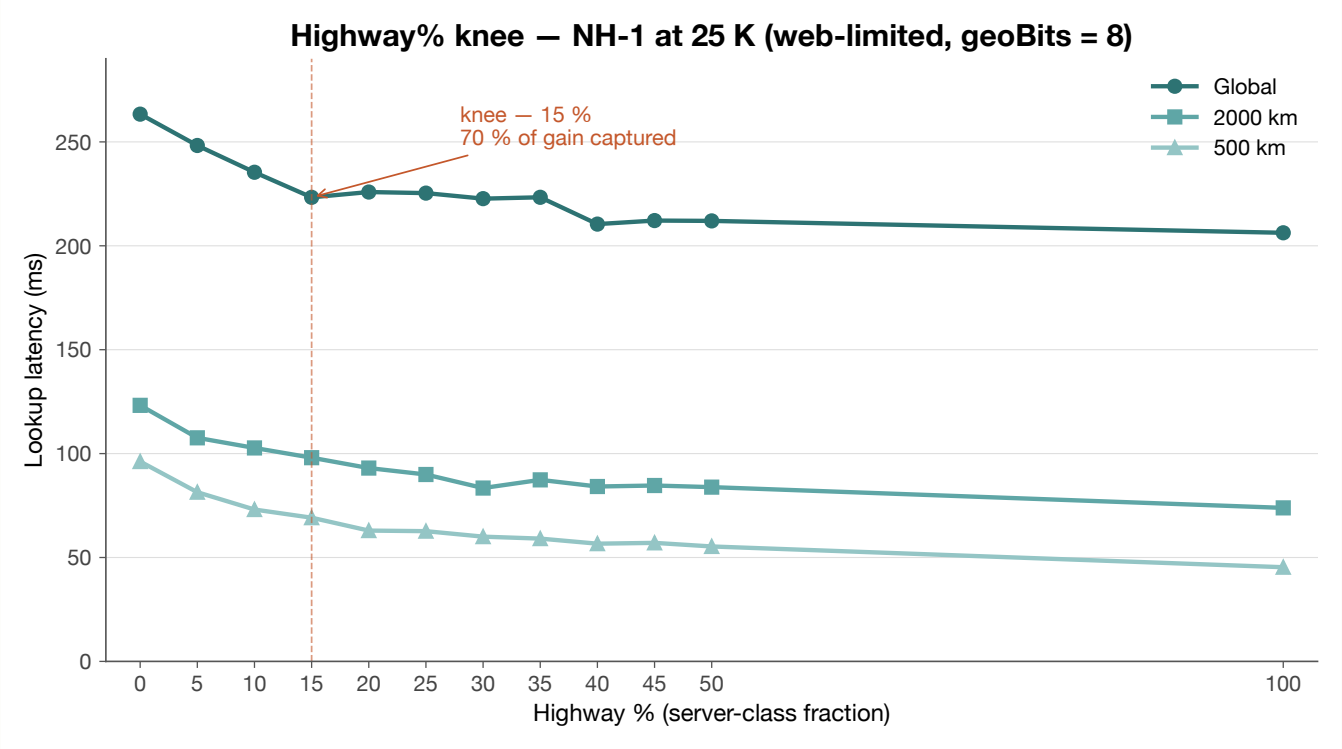
What this slide measures. Real P2P networks are heterogeneous: most participants run inside a browser (~50–100 connections, the WebRTC ceiling), but some run on real servers with effectively unlimited inbound capacity. We sweep the *fraction* of server-class nodes from 0 → 100 % and measure NH-1 latency at each point.

What "highway" means. A **highway node** is a server-class peer: `maxConnections = ∞`, synaptome cap raised from 50 to 256. It accepts unlimited inbound and acts as a transit hub. The rest of the network stays browser-class. Highway promotion is random — no privileged coordinator picks them — and the protocol treats highway and browser nodes identically apart from their cap.

Highway %	Global hops / ms	500 km ms	2000 km ms	Synaptome (highway)
0 (all browser)	5.09 / 263	96	123	n/a
5	4.61 / 248	81	108	217
15 (knee)	**4.16 / 223**	**69**	**98**	**218**
30	3.97 / 223	60	83	215
50	3.76 / 212	55	84	208
100 (all server)	3.52 / 206	45	74	244

15 % highway captures 70 % of available improvement *— a realistic deployment scenario where some peers run on powerful hardware.*

Highway %: the knee, charted



Three latency curves (global, 2000 km, 500 km) across the highway% sweep. The knee at 15 % is highlighted. Beyond 50 %, the system is essentially saturated — it has the routing it can use.

Pub/sub robustness

Metric	NH-1 at 25 K
Baseline delivery	100.0 %
Immediate (post-kill, no refresh)	99.9 %
Recovered (after 3 refresh rounds)	**100.0 %**
K-overlap (publisher ↔ subscriber views)	99.5 %
K-set stability (recovered, pub / sub)	95 % / 95 %
Dead-children	0
Orphans	0
Attached %	100 %

Same numbers across the entire highway% sweep — pub/sub robustness is independent of capacity asymmetry.

*The axonal tree heals through **routed re-subscribe**. There is no separate liveness ping, no parent tracking, no gossip.*

Honesty: locality vs. latency

N-DHT's priorities are speed and robustness — fast lookup, reliable delivery. Geographic location is *not a goal*; it is one input.

What the simulator does. Every protocol's hop latency is computed from physical (haversine) distance between simulated nodes — a faithful Internet model. This is what the real Internet looks like to all four protocols.

What the protocol does. A node knows *only its own* S2 cell — embedded once, at creation, into the top of its 64-bit ID:

```
nodeId = S2 cell prefix (8 bits) ‖ H(publicKey)
```

The protocol cannot ask another node *"where are you?"* — there is no such message. The S2 prefix creates **initial regional clustering at bootstrap** because nearby nodes are XOR-close, so a node's K-bucket and synaptome are seeded with mostly-local peers. After that, locality is a derived property of the routes that LTP and vitality reinforce.

Latency is used in routing, indirectly: AP scoring includes a `½^(latency_ms/100)` penalty, so observed RTT influences hop selection. Fast peers win at every distance — that is the speed-first priority in action.

The geoBits = 0 ablation

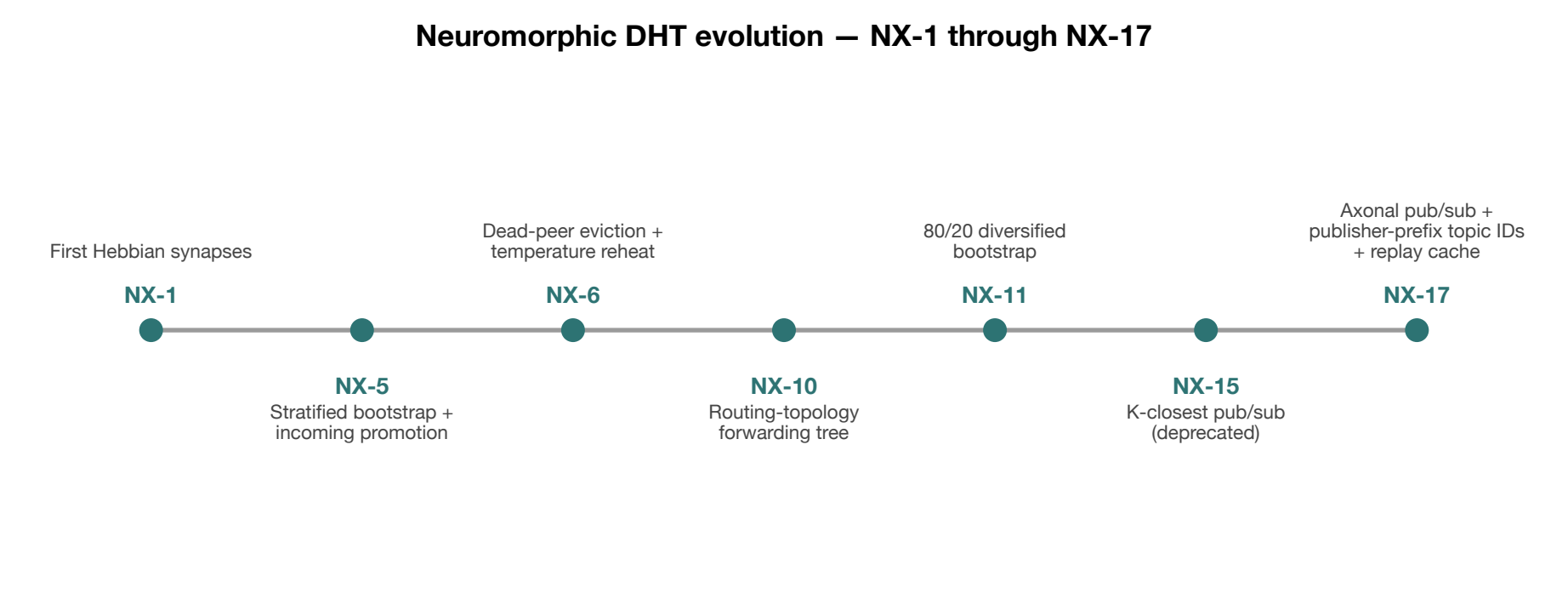
When we strip the S2 prefix and re-run, regional latency collapses:

Test	geoBits = 8	geoBits = 0	Δ
Global	5.09 / 263 ms	5.94 / 453 ms	+72 %
500 km	3.31 / 96 ms	**6.01 / 444 ms**	**+363 %**
2000 km	3.93 / 123 ms	6.01 / 437 ms	+255 %
5000 km	(n/a)	5.96 / 433 ms	flat

The latency penalty in AP routing is not strong enough, on its own, to discover locality from scratch within standard warmup (~5000 lookups). The S2 prefix is the **bootstrap shortcut**: it does not *teach* locality, it *seeds* the network with locality so reinforcement can sharpen routing within it.

Geography is an initial-condition shortcut for the latency optimisation, not a goal in itself. Future work: an embedded RTT-coordinate system (Vivaldi-style) could replace the S2 seed with a learned one — see the comparison slide later in this deck.

NX-1 → NX-17 — eight generations



Each generation addressed one concrete failure mode observed in its predecessor. NX-17 was the SOTA before NH-1.

NH-1 isn't a successor in that lineage — it's a *consolidation*. Same operations, fewer rules, single admission gate, ~270 lines vs NX-17's ~2300.

NX-17 rules and their NH-1 disposition

NX-17 rule	What it does	NH-1 status
Stratified bootstrap	Cold-start coverage across XOR strata	**Kept** (STRUCTURE)
Diversified bootstrap (80/20)	20 % random peers for global reach	**Kept** (merged into bootstrap)
Two-tier synaptome (50 + highway)	Separate "permanent" + "candidate" tiers	**Replaced** by per-node `_maxSynaptome`
Stratified eviction	Evict from over-represented stratum	**Replaced** by vitality eviction
Stratum floors	Guarantee min peers per stratum	**Removed** — vitality + diversity penalty handles it
Synaptome floor	Refuse eviction below N peers	**Removed** — vitality preserves locked entries
AP routing	Learned-weight greedy walk	**Kept** (NAVIGATE)
Two-hop lookahead	Probe second-hop alternatives	**Kept** (NAVIGATE)
Epsilon-greedy first hop	Random first hop with prob ε	**Kept** (EXPLORE)
Iterative fallback	k-closest fallback when AP fails	**Kept** (NAVIGATE)
LTP	Reinforce successful paths	**Kept** (LEARN)
Hop caching + lateral spread	Cache destination on intermediates	**Kept** (LEARN)
Triadic closure	Learn long edges from transit	**Kept** (LEARN)
Incoming promotion	Promote popular incoming peers	**Kept** (LEARN)
Adaptive decay	Per-node decay rate based on usage	**Default off** — flat γ = 0.995 wins
Simulated annealing	Replace lowest-vitality with 2-hop sample	**Kept** (EXPLORE)
Churn recovery (dead-syn eviction)	Drop dead peers on discovery	**Kept** (FORGET)
Temperature reheat on death	Spike T after dead-peer discovery	**Kept** (EXPLORE)
Diversity budget penalty	Penalise over-represented stratum groups	**Removed** v0.65.06 (harmful)
weightScale parameter	Bias AP scoring toward LTP weight	**Removed** v0.66.10 (no measurable effect)

NH-1 vs NX-17 — and why

Test	NX-17	NH-1	Δ (NH-1 vs NX-17)
Global	4.40 hops / 242 ms	5.15 hops / 263 ms	+17 % hops, +9 % ms
500 km	2.75 / 80 ms	3.28 / 95 ms	+19 % hops, +18 % ms
2000 km	3.27 / 105 ms	4.01 / 126 ms	+23 % hops, +20 % ms
5000 km	3.74 / 143 ms	4.54 / 169 ms	+21 % hops, +18 % ms
pubsubm delivered	100 %	100 %	tie
pubsubm + 5 % churn (recovered)	100 %	100 %	tie
dead-children / orphans	0 / 0	0 / 0	tie

The residual gap reflects two things:

1. NX-17's specialised eviction policies are tuned for the cap=100 case. Stratified eviction with stratum and synaptome floors *guarantees* a coverage profile that NH-1's vitality eviction approximates but doesn't pin. Under tight caps, that difference is measurable.

2. NH-1 trades peak performance for simplicity. ~270 lines vs NX-17's ~2300; 12 parameters vs ~44. The point is not that NH-1 is *faster* — it's that NH-1 is *almost as fast* with one unified admission gate that's easier to reason about and easier to extend.

*Highway% recovers the gap and more — at hw=15 % NH-1 is **223 ms global**, beating NX-17's all-browser 242 ms. The capped gap is the cost of consolidation. We accept it.*

Vivaldi vs N-DHT

Both are adaptive routing systems. They optimise different things.

Aspect	Vivaldi	N-DHT
Mechanism	Synthetic coordinates in N-D Euclidean space	Hebbian reinforcement on routing edges
What's learned	A position vector per node that predicts RTT to any peer	A weight per synapse, reinforced by traffic on successful paths
Output	RTT prediction (any pair)	A ranked list of next-hops (per lookup)
Locality discovery	Emergent from RTT measurements	Imposed via S2 cell ID prefix
Convergence guarantee	Yes — coordinate descent on RTT residuals	Empirical — depends on traffic mixing
Layered separation	Coordinate system is generic; any routing layer can use it	Routing and learning are integrated in a single protocol
Pub/sub support	Out of scope — Vivaldi is a primitive	First-class — axonal tree built on routed synaptome

Where they overlap: both treat the network as something to *measure and adapt to*, not a fixed topology to traverse.

Where they differ: Vivaldi predicts RTT, then any routing layer uses those predictions. N-DHT learns *paths* via reinforcement on the routing layer itself — locality is built into the IDs, learning sharpens the path within that locality.

Simulator integrity (v0.67)

The numbers in this deck are produced under explicit invariants:

1. **Bilateral connection cap honestly enforced.** Every node respects `connections.size ≤ maxConnections`. A base-class guard rail (`DHT.verifyConnectionCap`) runs at post-init, post-warmup, and after every churn round.
2. **Locality preserved.** No optimisation reads another node's routing table directly. Each node's `findKClosest` uses only its own state.
3. **Bounded RPC responses.** `findKClosest` simulates real Kademlia FIND_NODE with peer responses bounded to k=20.
4. **Per-protocol parameters honestly forwarded.** *(v0.67.01 — fixed a silent argument-drop in NH-1's `buildRoutingTables` that nullified `highwayPct` on all NH-1 runs. Prior highway% sweeps were retracted; this deck shows the corrected curve.)*
5. **Geographic prefix honestly configurable.** *(v0.67.01 — fixed a hardcoded `GEO_BITS=8` in NH-1 that ignored ablation requests. The geoBits=0 honesty slide above is the corrected measurement.)*

Every fix has its own commit and a research-log entry. Skepticism is part of the methodology, not a hindrance.

Transport layer

Transport-agnostic by design. One identity layer, three transports.

Transport	Role
WebRTC data channels	Browser-native; ~50 peers; NAT-traversing; DTLS-encrypted; signalling required
QUIC / TCP	Servers: unlimited peer count, lower overhead
WebSocket relay	Fallback for restrictive NATs; universal reachability, higher latency

Identity

Ed25519 keypair · `nodeId = cellPrefix ‖ H(pubkey)` · every control message signed.

Message protocol

Two layers. The application sees exactly two messages — SEND and RECEIVE. Everything else is the lower-level p2p wire protocol that implements them.

Application layer — what an application built on NH-1 actually calls:

Message	Purpose
SEND (target, payload)	Deliver a message toward a target — node, key, or topic
RECEIVE (handler)	Register to receive messages addressed here, or to a topic this node subscribes to

P2P wire layer — the messages NH-1 exchanges between peers to make SEND / RECEIVE work. Application code never constructs these directly.

Message	Purpose
PING / PONG	Liveness
FIND_NODE	DHT lookup step
ROUTE	Routed message toward a key (subscribe, publish, generic SEND)
SUBSCRIBE / UNSUBSCRIBE	Pub/sub membership; carries `lastSeenTs` for replay
PUBLISH	Publish to topic
DIRECT_DELIVER, ADOPT_SUBSCRIBERS, REPLAY_BATCH	Axonal pub/sub specifics

Common envelope (every wire message): version, type, sender, signature, timestamp, nonce, payload. Routing is **stateless in the protocol**. All state lives per-node: synaptome + axon roles.

Deployment considerations

- **Trust model.** No central authority. Public-key signatures authenticate peers.
- **Sybil resistance.** S2 cell prefix lightly discourages sybil swarms per-cell — not a full defense. Proof-of-location or join-PoW recommended for open deployments.
- **Provisioning.** Bootstrap via a small published sponsor set; fully decentralized thereafter.
- **Observability.** Nodes export local stats (synaptome health, LTP rate, role counts) for operator overlays.

Limitations and future directions

Known limitations

- **Warmup dependency.** N-DHT needs ~5 000 lookups before the synaptome converges. The first minute of deployment is suboptimal.
- **Synaptome capacity bounded at 50.** A deliberate cross-browser target. Server deployments could use 200–500.
- **Locality requires geographic IDs.** Without S2 prefix, regional locality collapses. *(Vivaldi-style RTT-only locality is future work.)*
- **Training is compute-optimising, not path-shortening.** Cannot close the bootstrap → omniscient hop gap.

Future directions

- **RTT-driven locality (Vivaldi integration).** Today's locality is structural; an embedded coordinate system could drive locality without a geo-prefix.
- **Adaptive synaptome capacity.** Bump capacity during join-heavy periods; prune in steady state.
- **Larger-scale evaluation.** 100 K / 250 K / 1 M nodes. Preliminary 50 K data exists; larger runs need server-side simulation.
- **Proof-of-location.** The S2 prefix is self-declared today. A verifiable location primitive would prevent prefix forgery.

Key takeaways

1. **NH-1 collapses ~18 NX-17 rules into 12 rules organised under 5 operations.** A single vitality function — `weight × recency` — drives every admission decision.
2. **15 % server-class nodes (highway%) captures 70 % of the available latency improvement.** Realistic deployment doesn't need a uniform server fleet.
3. **Pub/sub on routed axonal trees: 100 % baseline, 100 % recovered delivery under 5 % churn.** No K-closest replication, no gossip, no parent tracking — just routed re-subscribe.
4. **The simulator catches its own bugs.** Skepticism — "if it looks too good to be true, it is" — has saved this deck from at least three retracted findings during this session.
5. **Locality in NH-1 is structural (S2 prefix), not learned.** This is honest, comparable, and a pointed direction for future work (Vivaldi-style coordinate systems).

References

Whitepaper — `documents/Neuromorphic-DHT-Architecture.md` (this repository, v0.67)
Source + data — `github.com/YZ-social/dht-sim`

Architecture

- Saltzer, Reed, Clark · *End-to-End Arguments in System Design* (ACM TOCS 1984) — function placement principle
- Clark · *The Design Philosophy of the DARPA Internet Protocols* (SIGCOMM 1988)

Foundational DHT work

- Maymounkov & Mazières · *Kademlia: A Peer-to-peer Information System Based on the XOR Metric* (IPTPS 2002)
- Rowstron & Druschel · *Pastry: Scalable, decentralized object location and routing* (Middleware 2001)
- Zhao et al. · *Tapestry: A Resilient Global-Scale Overlay* (IEEE J-SAC 2004)

Adaptive coordinates

- Dabek, Cox, Kaashoek, Morris · ***Vivaldi: A Decentralized Network Coordinate System*** (SIGCOMM 2004)
- Cox, Dabek, Kaashoek, Li, Morris · *Practical, Distributed Network Coordinates* (HotNets 2003)
- Ledlie, Gardner, Seltzer · *Network Coordinates in the Wild* (NSDI 2007)

Substrate (learning, decay, pruning)

- Hebb · *The Organization of Behavior* (1949) — synaptic potentiation
- Ebbinghaus · *Über das Gedächtnis* (1885) — exponential forgetting curve
- Frey & Morris · *Synaptic tagging and the late phase of LTP* (Nature 1997) — biological analog of weight × recency retention
- LeCun, Denker, Solla · *Optimal Brain Damage* (NeurIPS 1989) — prune lowest-magnitude connections
- O'Neil, O'Neil, Weikum · *The LRU-K Page Replacement Algorithm* (SIGMOD 1993) — multi-history recency for cache replacement
- Watts & Strogatz · *Collective Dynamics of Small-World Networks* (Nature 1998)
- Google · *S2 Geometry Library* (2011)

Questions

github.com/YZ-social/dht-sim · v0.67.02